A3-R.R

ujjwa

2025-06-26

```
# Set the base directory for the project
BASE <- "C:\\Users\\ujjwa\\Documents\\VCU\\Pre-Course\\SCMA632\\Assignments\\A3\\R"
setwd(BASE)  # Change working directory to where your data files are located
getwd()      # Confirm working directory
```

```
## [1] "C:/Users/ujjwa/Documents/VCU/Pre-Course/SCMA632/Assignments/A3/R"
```

```
# Define a helper function to install packages if not already installed
install <- function(pkg) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, quiet = TRUE)
  }
}

# Define a helper function to load packages
load <- function(pkg) {
  library(pkg, character.only = TRUE, quietly = TRUE)
}

# Required packages for this analysis
pkgs <- c("tidyverse", "readxl", "car", "stringdist", "lubridate", "stats", "fitdistrplus")
lapply(pkgs, install)
```

```
## Loading required package: tidyverse
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: readxl
##
## Loading required package: car
##
## Loading required package: carData
```

```
##
##
## Attaching package: 'car'
##
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
##
## The following object is masked from 'package:purrr':
##
##     some
##
##
## Loading required package: stringdist
##
##
## Attaching package: 'stringdist'
##
##
## The following object is masked from 'package:tidyr':
##
##     extract
##
##
## Loading required package: fitdistrplus
##
## Loading required package: MASS
##
##
## Attaching package: 'MASS'
##
##
## The following object is masked from 'package:dplyr':
##
##     select
##
##
## Loading required package: survival

## [[1]]
## NULL
##
## [[2]]
## NULL
##
## [[3]]
## NULL
##
## [[4]]
## NULL
##
## [[5]]
```

```
## NULL
##
## [[6]]
## NULL
##
## [[7]]
## NULL
```

```
lapply(pkgs, load)
```

```
## [[1]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[2]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[3]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[4]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[5]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[6]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
##
## [[7]]
##  [1] "fitdistrplus" "survival"     "MASS"         "stringdist"   "car"
```

```
##  [6] "carData"      "readxl"       "lubridate"    "forcats"      "stringr"
## [11] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [16] "ggplot2"      "tidyverse"    "stats"        "graphics"     "grDevices"
## [21] "utils"        "datasets"     "methods"      "base"
```

```
# --- Part (a): NSSO68 Dataset Analysis ---

# Load the NSSO68 dataset
df_001 <- read.csv(file.path(BASE, "datasets", "NSSO68.csv"))

# Display unique states to verify data loading
print(unique(df_001$state_1))
```

```
##  [1] "GUJ"    "ORI"    "CHTSD"  "MP"     "JRKD"   "WB"     "AP"     "MH"
##  [9] "D&D"    "D&NH"   "MIZ"    "TRPR"   "MANPR"  "ASSM"   "MEG"    "NAG"
## [17] "A&N"    "PNDCRY" "TN"     "GOA"    "KA"     "KE"     "LKSDP"  "SKM"
## [25] "Bhr"    "UP"     "RJ"     "ARP"    "DL"     "HR"     "Pun"    "HP"
## [33] "UT"     "Chandr" "J$K"
```

```
# Filter data for "MEG" state and select relevant columns for analysis
df_001 <- df_001 %>%
  dplyr::filter(state_1 == "MEG") %>%
  dplyr::select(foodtotal_q, MPCE_MRP, Age, Meals_At_Home, Possess_ration_card, Education, No_of_Meals_

# Display DataFrame information (summary and structure)
str(df_001)
```

```
## 'data.frame':    1259 obs. of  7 variables:
##  $ foodtotal_q        : num  29.9 30.1 24.8 23.2 19.7 ...
##  $ MPCE_MRP           : num  3186 3939 2370 2203 2372 ...
##  $ Age                : int  47 56 70 58 42 45 27 44 72 32 ...
##  $ Meals_At_Home      : int  60 60 60 60 60 60 60 60 60 60 ...
##  $ Possess_ration_card: int  1 1 1 1 1 1 1 2 2 2 ...
##  $ Education          : int  10 10 5 10 12 12 10 2 5 10 ...
##  $ No_of_Meals_per_day: int  2 2 2 2 2 2 2 2 2 2 ...
```

```
summary(df_001)
```

```
##   foodtotal_q       MPCE_MRP            Age        Meals_At_Home
##  Min.   : 0.00   Min.   :  591.8   Min.   : 9.00   Min.   : 0.00
##  1st Qu.:15.76   1st Qu.: 1185.8   1st Qu.:37.00   1st Qu.:58.00
##  Median :18.30   Median : 1523.1   Median :45.00   Median :60.00
##  Mean   :18.89   Mean   : 1809.1   Mean   :44.92   Mean   :58.09
##  3rd Qu.:21.32   3rd Qu.: 2130.3   3rd Qu.:52.00   3rd Qu.:60.00
##  Max.   :41.58   Max.   :29146.1   Max.   :90.00   Max.   :90.00
##                                                    NA's   :8
##  Possess_ration_card   Education      No_of_Meals_per_day
##  Min.   :1.00        Min.   : 1.000   Min.   :0.000
##  1st Qu.:1.00        1st Qu.: 6.000   1st Qu.:2.000
##  Median :1.00        Median : 7.000   Median :2.000
##  Mean   :1.37        Mean   : 7.318   Mean   :1.999
##  3rd Qu.:2.00        3rd Qu.:10.000   3rd Qu.:2.000
```

```
##  Max.   :2.00        Max.   :13.000   Max.   :3.000
##                                       NA's   :1
```

```r
# Impute missing values with the mean for specified columns
impute_with_mean <- function(df, cols_to_impute) {
  for (col in cols_to_impute) {
    if (col %in% colnames(df)) {
      df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
    }
  }
  return(df)
}

cols_to_impute <- c('Meals_At_Home', 'No_of_Meals_per_day')
df_001 <- impute_with_mean(df_001, cols_to_impute)

# Replace infinite values (positive or negative) with NA
df_001[sapply(df_001, is.infinite)] <- NA

# Check for any remaining missing values after imputation and replacement
print(colSums(is.na(df_001)))
```

```
##         foodtotal_q            MPCE_MRP                 Age       Meals_At_Home
##                   0                   0                   0                   0
## Possess_ration_card           Education No_of_Meals_per_day
##                   0                   0                   0
```

```r
# Convert 'Possess_ration_card' to factor if it's categorical
df_001$Possess_ration_card <- as.factor(df_001$Possess_ration_card)

# Create and fit the Ordinary Least Squares (OLS) regression model
model_nsso <- lm(foodtotal_q ~ MPCE_MRP + Age + Meals_At_Home + Possess_ration_card + Education + No_of_

# Print the summary of the regression results
print(summary(model_nsso))
```

```
##
## Call:
## lm(formula = foodtotal_q ~ MPCE_MRP + Age + Meals_At_Home + Possess_ration_card +
##     Education + No_of_Meals_per_day, data = df_001)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -33.365  -2.614  -0.301   2.305  20.705
##
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)           9.128619   2.367551   3.856 0.000121 ***
## MPCE_MRP              0.001245   0.000107  11.632  < 2e-16 ***
## Age                   0.052218   0.010409   5.016 6.02e-07 ***
## Meals_At_Home         0.180090   0.024098   7.473 1.46e-13 ***
## Possess_ration_card2  0.115067   0.262421   0.438 0.661112
## Education             0.207096   0.048886   4.236 2.44e-05 ***
```

```
## No_of_Meals_per_day  -3.427736   1.042416  -3.288 0.001036 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.216 on 1252 degrees of freedom
## Multiple R-squared:  0.1915, Adjusted R-squared:  0.1876
## F-statistic: 49.41 on 6 and 1252 DF,  p-value: < 2.2e-16
```

```
# Calculate Variance Inflation Factor (VIF) to check for multicollinearity
vif_results_nsso <- vif(model_nsso)
print(vif_results_nsso)
```

```
##            MPCE_MRP                 Age       Meals_At_Home Possess_ration_card
##            1.199202            1.052001            1.065220            1.137026
##           Education No_of_Meals_per_day
##            1.256799            1.039085
```

```
# Extract and print the coefficients of the fitted model
coeff_nsso <- coef(model_nsso)
print(coeff_nsso)
```

```
##          (Intercept)             MPCE_MRP                  Age
##          9.128618843         0.001244749         0.052217767
##        Meals_At_Home Possess_ration_card2           Education
##          0.180090070         0.115066988         0.207096518
##  No_of_Meals_per_day
##         -3.427736069
```

```
# Construct and print the regression equation
eq_nsso <- paste0("y = ", round(coeff_nsso[1], 2))
for (i in 2:length(coeff_nsso)) {
  eq_nsso <- paste0(eq_nsso, " + ", round(coeff_nsso[i], 6), "*", names(coeff_nsso)[i])
}
print(eq_nsso)
```

```
## [1] "y = 9.13 + 0.001245*MPCE_MRP + 0.052218*Age + 0.18009*Meals_At_Home + 0.115067*Possess_ration_ca
```

```
# Delete the DataFrame to free up memory
rm(df_001)


# --- Part (b): IPL Player Performance and Salary Analysis ---

# Define file paths
perf_path <- file.path(BASE, "datasets", "IPL_ball_by_ball_updated till 2024.csv")
salary_path <- file.path(BASE, "datasets", "IPL SALARIES 2024.xlsx")

# Read the datasets
perf_df <- read_csv(perf_path)
```

```
## Rows: 255759 Columns: 19
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (12): Date, Season, Batting team, Bowling team, Bowler, Striker, Non Str...
## dbl  (7): Match id, Innings No, Ball No, runs_scored, extras, score, wicket_...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
sal_df <- read_excel(salary_path)

# Select only relevant columns and extract the year from the date column
columns_to_select <- c("Match id", "Date", "Season", "Innings No",
                       "Bowler", "Striker", "runs_scored", "wicket_confirmation")
perf_df <- perf_df %>% dplyr::select(all_of(columns_to_select))
# Using dmy() from lubridate for consistent date parsing
perf_df$Year <- year(dmy(perf_df$Date))

# --- Aggregate Data ---

# Summarize total runs by each batsman
runs_agg <- perf_df %>%
  group_by(Year, `Innings No`, Striker) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

# Summarize total wickets by each bowler
wickets_agg <- perf_df %>%
  group_by(Year, `Innings No`, Bowler) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

# --- Identify Top 3 Performers Each Year ---

years_ipl <- unique(runs_agg$Year)
for (yr in years_ipl) {
  cat("Year:", yr, "\n\nTop 3 Run Scorers:\n")
  print(runs_agg %>% dplyr::filter(Year == yr) %>%
          group_by(Striker) %>%
          summarise(runs = sum(runs_scored), .groups = 'drop') %>%
          arrange(desc(runs)) %>% head(3))

  cat("\nTop 3 Wicket Takers:\n")
  print(wickets_agg %>% dplyr::filter(Year == yr) %>%
          group_by(Bowler) %>%
          summarise(wickets = sum(wicket_confirmation), .groups = 'drop') %>%
          arrange(desc(wickets)) %>% head(3))

  cat("\n", strrep("=", 50), "\n\n")
}
```

```
## Year: 2008
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker        runs
##   <chr>         <dbl>
```

```
## 1 SE Marsh        616
## 2 G Gambhir       534
## 3 ST Jayasuriya   514
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler        wickets
##   <chr>           <dbl>
## 1 Sohail Tanvir      24
## 2 IK Pathan          20
## 3 JA Morkel          20
##
##  ================================================
##
## Year: 2009
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker         runs
##   <chr>          <dbl>
## 1 ML Hayden        572
## 2 AC Gilchrist     495
## 3 AB de Villiers   465
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler   wickets
##   <chr>      <dbl>
## 1 RP Singh      26
## 2 A Kumble      22
## 3 A Nehra       22
##
##  ================================================
##
## Year: 2010
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker       runs
##   <chr>        <dbl>
## 1 SR Tendulkar   618
## 2 JH Kallis      572
## 3 SK Raina       528
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler          wickets
##   <chr>             <dbl>
## 1 PP Ojha              22
## 2 A Mishra             20
## 3 Harbhajan Singh      20
##
##  ================================================
##
```

```
## Year: 2011
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker       runs
##   <chr>        <dbl>
## 1 CH Gayle       608
## 2 V Kohli        557
## 3 SR Tendulkar   553
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler     wickets
##   <chr>        <dbl>
## 1 SL Malinga      30
## 2 MM Patel        22
## 3 S Aravind       22
##
##  ================================================
##
## Year: 2012
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker    runs
##   <chr>     <dbl>
## 1 CH Gayle    733
## 2 G Gambhir   590
## 3 S Dhawan    569
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler     wickets
##   <chr>        <dbl>
## 1 M Morkel        30
## 2 SP Narine       29
## 3 SL Malinga      25
##
##  ================================================
##
## Year: 2013
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker     runs
##   <chr>      <dbl>
## 1 MEK Hussey   733
## 2 CH Gayle     720
## 3 V Kohli      639
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler        wickets
##   <chr>           <dbl>
```

```
## 1 DJ Bravo           34
## 2 JP Faulkner        33
## 3 R Vinay Kumar      27
##
##  ================================================
##
## Year: 2014
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker     runs
##   <chr>      <dbl>
## 1 RV Uthappa   660
## 2 DR Smith     566
## 3 GJ Maxwell   552
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler    wickets
##   <chr>       <dbl>
## 1 MM Sharma      26
## 2 SP Narine      22
## 3 B Kumar        21
##
##  ================================================
##
## Year: 2015
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker      runs
##   <chr>       <dbl>
## 1 DA Warner     562
## 2 AM Rahane     540
## 3 LMP Simmons   540
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler     wickets
##   <chr>        <dbl>
## 1 DJ Bravo        28
## 2 SL Malinga      26
## 3 A Nehra         25
##
##  ================================================
##
## Year: 2016
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker         runs
##   <chr>          <dbl>
## 1 V Kohli          973
## 2 DA Warner        848
```

```
## 3 AB de Villiers   687
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler    wickets
##   <chr>       <dbl>
## 1 B Kumar        24
## 2 SR Watson      23
## 3 YS Chahal      22
##
##  ================================================
##
## Year: 2017
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker    runs
##   <chr>     <dbl>
## 1 DA Warner   641
## 2 G Gambhir   498
## 3 S Dhawan    479
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler     wickets
##   <chr>        <dbl>
## 1 B Kumar         28
## 2 JD Unadkat      27
## 3 JJ Bumrah       23
##
##  ================================================
##
## Year: 2018
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker        runs
##   <chr>         <dbl>
## 1 KS Williamson   735
## 2 RR Pant         684
## 3 KL Rahul        659
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler      wickets
##   <chr>         <dbl>
## 1 AJ Tye           28
## 2 S Kaul           24
## 3 Rashid Khan      23
##
##  ================================================
##
## Year: 2019
##
```

```
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker    runs
##   <chr>     <dbl>
## 1 DA Warner   692
## 2 KL Rahul    593
## 3 Q de Kock   529
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler      wickets
##   <chr>         <dbl>
## 1 K Rabada         29
## 2 Imran Tahir      26
## 3 JJ Bumrah        23
##
##  ================================================
##
## Year: 2020
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker    runs
##   <chr>     <dbl>
## 1 KL Rahul    676
## 2 S Dhawan    618
## 3 DA Warner   548
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler    wickets
##   <chr>       <dbl>
## 1 K Rabada       32
## 2 JJ Bumrah      30
## 3 TA Boult       26
##
##  ================================================
##
## Year: 2021
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker       runs
##   <chr>        <dbl>
## 1 RD Gaikwad     635
## 2 F du Plessis   633
## 3 KL Rahul       626
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler     wickets
##   <chr>        <dbl>
## 1 HV Patel        35
## 2 Avesh Khan      27
```

```
## 3 JJ Bumrah       22
##
##  ================================================
##
## Year: 2022
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker     runs
##   <chr>      <dbl>
## 1 JC Buttler   863
## 2 KL Rahul     616
## 3 Q de Kock    508
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler       wickets
##   <chr>          <dbl>
## 1 YS Chahal         29
## 2 PWH de Silva      27
## 3 K Rabada          23
##
##  ================================================
##
## Year: 2023
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker       runs
##   <chr>        <dbl>
## 1 Shubman Gill   890
## 2 F du Plessis   730
## 3 DP Conway      672
##
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler         wickets
##   <chr>            <dbl>
## 1 MM Sharma           31
## 2 Mohammed Shami      28
## 3 Rashid Khan         28
##
##  ================================================
##
## Year: 2024
##
## Top 3 Run Scorers:
## # A tibble: 3 x 2
##   Striker          runs
##   <chr>           <dbl>
## 1 RD Gaikwad        509
## 2 V Kohli           500
## 3 B Sai Sudharsan   418
##
```

```
## Top 3 Wicket Takers:
## # A tibble: 3 x 2
##   Bowler         wickets
##   <chr>            <dbl>
## 1 HV Patel            19
## 2 Mukesh Kumar        15
## 3 Arshdeep Singh      14
##
##  ================================================
```

```r
# --- Name Matching Between Datasets (Fuzzy Matching) ---

# Using the provided match_names function with jw distance
match_names <- function(name, choices, threshold = 0.2) {
  if (is.na(name)) return(NA)
  dists <- stringdist(name, choices, method = "jw")
  min_dist <- min(dists)
  if (min_dist <= threshold) return(choices[which.min(dists)])
  return(NA)
}

# --- Correlation Between Salary and Performance (2024) ---

runs_2024 <- perf_df %>%
  dplyr::filter(Year == 2024) %>%
  group_by(Striker) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

wickets_2024 <- perf_df %>%
  dplyr::filter(Year == 2024) %>%
  group_by(Bowler) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

sal_df$Matched_Striker <- sapply(sal_df$Player, match_names, choices = runs_2024$Striker)
sal_df$Matched_Bowler <- sapply(sal_df$Player, match_names, choices = wickets_2024$Bowler)

# Merge salary with runs and wickets data for 2024
striker_merged <- merge(sal_df, runs_2024, by.x = "Matched_Striker", by.y = "Striker", all.x = TRUE)
bowler_merged <- merge(sal_df, wickets_2024, by.x = "Matched_Bowler", by.y = "Bowler", all.x = TRUE)

# Calculate combined performance points for 2024 for players who have both runs and wickets
# First, create a base dataframe with unique players and their salaries
combined_perf_2024 <- sal_df %>%
  dplyr::select(Player, Rs) %>%
  dplyr::rename(Full_Name = Player, Salary = Rs)

# Join with runs data
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::left_join(runs_2024 %>% dplyr::rename(Full_Name = Striker), by = c("Full_Name")) %>%
  dplyr::rename(Runs_2024 = runs_scored)

# Join with wickets data
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::left_join(wickets_2024 %>% dplyr::rename(Full_Name = Bowler), by = c("Full_Name")) %>%
```

```r
  dplyr::rename(Wickets_2024 = wicket_confirmation)

# Fill NA for runs and wickets with 0 for calculation
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::mutate(
    Runs_2024 = replace_na(Runs_2024, 0),
    Wickets_2024 = replace_na(Wickets_2024, 0)
  )

# Calculate Total Points (adjust multiplier as needed, e.g., 25 for wickets)
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::mutate(Total_Points_2024 = Runs_2024 + (Wickets_2024 * 25))

# Filter for players with valid salary and at least some performance data for correlation
cor_data_2024 <- combined_perf_2024 %>%
  dplyr::filter(!is.na(Salary) & Total_Points_2024 > 0)

cor_total_points_salary <- cor(cor_data_2024$Salary, cor_data_2024$Total_Points_2024, use = "complete.ob

cat("\nCorrelation between Salary and Runs in 2024 (using merged data):", cor(striker_merged$Rs, striker
```

```
##
## Correlation between Salary and Runs in 2024 (using merged data): 0.4531945
```

```r
cat("\nCorrelation between Salary and Wickets in 2024 (using merged data):", cor(bowler_merged$Rs, bowle
```

```
##
## Correlation between Salary and Wickets in 2024 (using merged data): 0.2137848
```

```r
cat("\nCorrelation between Salary and Total Points in 2024:", cor_total_points_salary)
```

```
##
## Correlation between Salary and Total Points in 2024: 0.3384091
```

```r
# Prepare data for linear regression: 'Total Points' as independent variable (X) and 'Salary' as depend
# Using the 2024 combined performance data for this
y_ipl <- cor_data_2024$Salary
X_ipl <- cor_data_2024 %>% dplyr::select(Total_Points_2024)

# Split the data into training and testing sets
set.seed(42) # For reproducibility
train_indices_ipl <- sample(1:nrow(cor_data_2024), size = 0.75 * nrow(cor_data_2024))
X_train_ipl <- cor_data_2024[train_indices_ipl, 'Total_Points_2024', drop = FALSE]
# Explicitly extract the 'Salary' column as a vector
y_train_ipl <- cor_data_2024[train_indices_ipl, 'Salary'][[1]]
X_test_ipl <- cor_data_2024[-train_indices_ipl, 'Total_Points_2024', drop = FALSE]
# Explicitly extract the 'Salary' column as a vector
y_test_ipl <- cor_data_2024[-train_indices_ipl, 'Salary'][[1]]

# Initialize and fit the Linear Regression model
model_ipl <- lm(Salary ~ Total_Points_2024, data = data.frame(Salary = y_train_ipl, Total_Points_2024 =
```

```r
# Make predictions
y_pred_ipl <- predict(model_ipl, newdata = data.frame(Total_Points_2024 = X_test_ipl$Total_Points_2024))

# Evaluate the model (R-squared and Mean Squared Error)
summary_model_ipl <- summary(model_ipl)
r2_ipl <- summary_model_ipl$r.squared
mse_ipl <- mean((y_test_ipl - y_pred_ipl)^2)

print(paste("R-squared (Training):", r2_ipl))
```

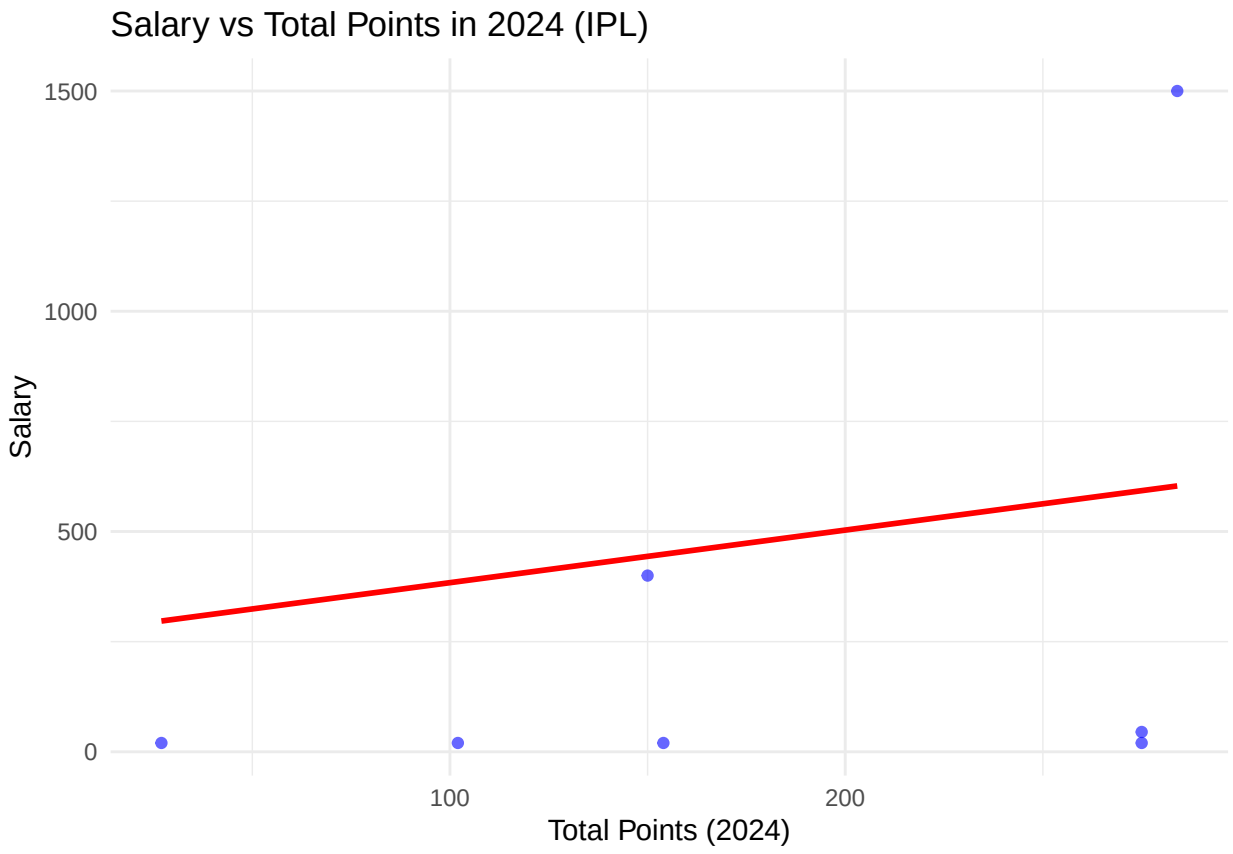
```
## [1] "R-squared (Training): 0.0936862354079638"
```

```r
print(paste("Mean Squared Error (Test):", mse_ipl))
```

```
## [1] "Mean Squared Error (Test): 261135.309686655"
```

```r
# Plotting the regression results
plot_df <- data.frame(Total_Points_2024 = X_test_ipl$Total_Points_2024, Actual_Salary = y_test_ipl, Pred

ggplot(plot_df, aes(x = Total_Points_2024)) +
  geom_point(aes(y = Actual_Salary), color = 'blue', alpha = 0.6) + # Removed redundant label
  geom_line(aes(y = Predicted_Salary), color = 'red', linewidth = 1) + # Removed redundant label
  labs(x = 'Total Points (2024)', y = 'Salary', title = 'Salary vs Total Points in 2024 (IPL)') +
  theme_minimal()
```



```r
# --- Distribution Fitting for Assigned Player: N Pooran ---

n_pooran_runs <- perf_df %>%
  group_by(Year, Striker, `Match id`) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop') %>%
  dplyr::filter(Striker == "N Pooran") %>%
  pull(runs_scored)

n_pooran_runs_pos <- n_pooran_runs[n_pooran_runs > 0]

fit_norm <- tryCatch(fitdist(n_pooran_runs, "norm"), error = function(e) NULL)
fit_gamma <- tryCatch(fitdist(n_pooran_runs, "gamma"), error = function(e) NULL)
fit_exp <- tryCatch(fitdist(n_pooran_runs, "exp"), error = function(e) NULL)
fit_lnorm <- tryCatch(fitdist(n_pooran_runs_pos, "lnorm"), error = function(e) NULL)

# Filter out NULL fits for gofstat
fits_pooran <- list(norm = fit_norm, gamma = fit_gamma, exp = fit_exp)
fits_pooran <- Filter(Negate(is.null), fits_pooran)

gof <- if(length(fits_pooran) > 0) tryCatch(gofstat(fits_pooran), error = function(e) NULL) else NULL
gof_lnorm <- if (!is.null(fit_lnorm) && length(n_pooran_runs_pos) > 0 && all(n_pooran_runs_pos > 0)) try


print(gof)
```

```
## Goodness-of-fit statistics
##                               1-mle-norm 2-mle-gamma 3-mle-exp
## Kolmogorov-Smirnov statistic  0.1371003   0.1734986 0.1458793
## Cramer-von Mises statistic    0.2659139   0.4227711 0.2878949
## Anderson-Darling statistic    1.7800216         Inf       Inf
##
## Goodness-of-fit criteria
##                                 1-mle-norm 2-mle-gamma 3-mle-exp
## Akaike's Information Criterion   605.0947    513.6439  565.3804
## Bayesian Information Criterion   609.5337    518.0829  567.5999
```

```r
cat("\nLognormal fit (positive data only):\n")
```

```
##
## Lognormal fit (positive data only):
```

```r
print(gof_lnorm)
```

```
## NULL
```

```r
# Perform KS test for Gamma fit if it was successful
if (!is.null(fit_gamma)) {
  ks <- ks.test(n_pooran_runs, "pgamma", shape = fit_gamma$estimate["shape"], rate = fit_gamma$estimate
  print(ks)
} else {
  cat("Gamma fit not available for KS test.\n")
}
```

```
## Warning in ks.test.default(n_pooran_runs, "pgamma", shape =
## fit_gamma$estimate["shape"], : ties should not be present for the one-sample
## Kolmogorov-Smirnov test

##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  n_pooran_runs
## D = 0.1735, p-value = 0.03335
## alternative hypothesis: two-sided
```

```
# --- Distribution Fitting for Top 3 Batsmen and Bowlers (Last 3 Seasons) ---

# Reusing the provided get_best_fit function
get_best_fit <- function(data) {
  if (length(data) < 2 || all(is.na(data))) return(list(best_dist = "Too few data or all NA", gof = NULL
  data_pos <- data[data > 0]
  use_pos_only <- !any(data <= 0) # True if all data is positive

  fits <- list(
    norm = tryCatch(fitdist(data, "norm"), error = function(e) NULL),
    gamma = tryCatch(fitdist(data_pos, "gamma"), error = function(e) NULL), # Gamma requires positive d
    exp = tryCatch(fitdist(data_pos, "exp"), error = function(e) NULL)     # Exponential requires posit
  )

  if (length(data_pos) > 0) {
    fits$lnorm <- tryCatch(fitdist(data_pos, "lnorm"), error = function(e) NULL)
  }

  fits <- Filter(Negate(is.null), fits)

  if (length(fits) == 0) return(list(best_dist = "No suitable fit", gof = NULL))
  if (length(fits) == 1) return(list(best_dist = names(fits), gof = NULL))

  # Use AIC to select the best fit if multiple are available
  aics <- sapply(fits, function(f) f$aic)
  best_dist_name <- names(which.min(aics))

  return(list(best_dist = best_dist_name, gof = NULL)) # Only returning best_dist as per original struc
}


last_3_seasons <- sort(unique(perf_df$Year), decreasing = TRUE)[1:3]

df_runs_match_level <- perf_df %>%
  group_by(Year, Striker, `Match id`) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

df_wickets_match_level <- perf_df %>%
  group_by(Year, Bowler, `Match id`) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

for (year in last_3_seasons) {
  cat("\n===== Year:", year, "=====")
```

```r
  top_batsmen <- df_runs_match_level %>%
    dplyr::filter(Year == year) %>%
    group_by(Striker) %>%
    summarise(total_runs = sum(runs_scored), .groups = 'drop') %>%
    arrange(desc(total_runs)) %>%
    slice(1:3) %>% pull(Striker)

  top_bowlers <- df_wickets_match_level %>%
    dplyr::filter(Year == year) %>%
    group_by(Bowler) %>%
    summarise(total_wickets = sum(wicket_confirmation), .groups = 'drop') %>%
    arrange(desc(total_wickets)) %>%
    slice(1:3) %>% pull(Bowler)

  cat("\nTop 3 Batsmen Distribution Fits:\n")
  for (batsman in top_batsmen) {
    player_data <- df_runs_match_level %>% dplyr::filter(Year == year, Striker == batsman) %>% pull(runs
    result <- get_best_fit(player_data)
    cat("-", batsman, ": Best Fit:", result$best_dist, "\n")
  }

  cat("\nTop 3 Bowlers Distribution Fits:\n")
  for (bowler in top_bowlers) {
    player_data <- df_wickets_match_level %>% dplyr::filter(Year == year, Bowler == bowler) %>% pull(wi
    result <- get_best_fit(player_data)
    cat("-", bowler, ": Best Fit:", result$best_dist, "\n")
  }
}
```

```
##
## ===== Year: 2024 =====
## Top 3 Batsmen Distribution Fits:
## - RD Gaikwad : Best Fit: exp
## - V Kohli : Best Fit: exp
## - B Sai Sudharsan : Best Fit: gamma
##
## Top 3 Bowlers Distribution Fits:
## - HV Patel : Best Fit: lnorm
## - Mukesh Kumar : Best Fit: lnorm
## - Arshdeep Singh : Best Fit: lnorm
##
## ===== Year: 2023 =====
## Top 3 Batsmen Distribution Fits:
## - Shubman Gill : Best Fit: gamma
## - F du Plessis : Best Fit: norm
## - DP Conway : Best Fit: exp
##
## Top 3 Bowlers Distribution Fits:
## - MM Sharma : Best Fit: gamma
## - Mohammed Shami : Best Fit: lnorm
## - Rashid Khan : Best Fit: lnorm
##
## ===== Year: 2022 =====
```

```
## Top 3 Batsmen Distribution Fits:
## - JC Buttler : Best Fit: exp
## - KL Rahul : Best Fit: exp
## - Q de Kock : Best Fit: exp
##
## Top 3 Bowlers Distribution Fits:
## - YS Chahal : Best Fit: lnorm
## - PWH de Silva : Best Fit: lnorm
## - K Rabada : Best Fit: lnorm
```